```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

int main() {
    int fd[2];
    char write_msg[100];
    char read_msg[100];

    if (pipe(fd) == -1) {
        perror("Pipe failed");
        exit(1);
    }

    pid_t pid;
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(1);
    }

    // Parent Process
    if (pid > 0) {
        close(fd[0]); // Close read end
        printf("Enter message: ");
        scanf("%s", write_msg);
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]);
    }

    // Child Process
    else {
        close(fd[1]); // Close write end
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]);
    }
}
############################
#include <stdio.h>
#include <stdlib.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    int shmid;
    shmid = shmget(2345, 1024, 0666 | IPC_CREAT);
    if (shmid == -1) {
        perror("shmget");
        exit(1);
    }

        char *shared_memory;
    shared_memory = (char *)shmat(shmid, NULL, 0);

    printf("Shared Memory ID : %d\n", shmid);
    printf("Process attached at %p\n", shared_memory);

    printf("Enter message : ");

    char message[100];
    fgets(message, sizeof(message), stdin);

    strcpy(shared_memory, message);

    printf("Written to Shared Memory : %s", shared_memory);
}
```